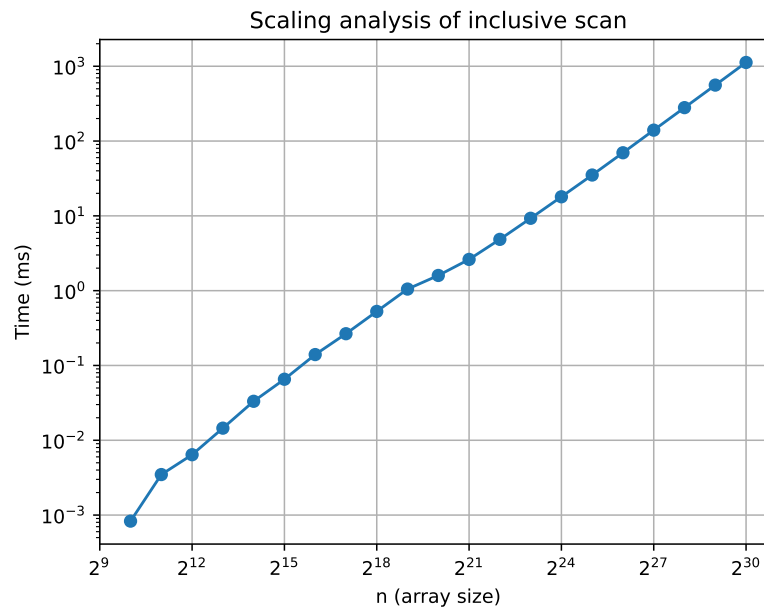


GitHub Link: <https://github.com/pmehrotra3/repo759/HW02>

Task 1.

- a) `scan.cpp` present in HW02 folder.
- b) `task1.cpp` present in HW02 folder.
- c) The plot of the time taken by `scan` versus n is shown below.



Task 2.

- a) `convolution.cpp` present in HW02 folder.
- b) `task2.cpp` present in HW02 folder.

Task 3.

- a)–d) `matmul.cpp` (containing `mmul1`, `mmul2`, `mmul3` and `mmul4`) present in HW02 folder.
- e) `task3.cpp` present in HW02 folder.

- f) On Euler with $n = 1024$, `mmul2` was the fastest (126 ms), `mmul1` was slower (6326 ms), and `mmul3` was the slowest (18208 ms). This is due to cache locality: since matrices are stored in row-major order, accessing along a row uses contiguous memory, while accessing down a column skips n elements each step and causes cache misses. The inner loop of `mmul2` accesses rows, `mmul1` accesses one column, and `mmul3` accesses two columns. `mmul1` and `mmul4` (6807 ms) perform nearly the same because they share the same loop order, and `std::vector` stores its elements contiguously like an array.